

Шпаргалка по Redis

Подключение, структуры данных и команды для них, сроки жизни ключей, публикация и подписка, транзакции, сохранение на диск, наблюдение за сервером и типовые приёмы.

Подключение

- redis-cli
 - Клиент на localhost:6379
- redis-cli -h host -p 6379 -a пароль -n 2
 - Узел, порт, пароль и номер базы
- redis-cli --scan --pattern "user:*"
 - Постраничный обход ключей по маске
- PING
 - Проверка связи, ответ PONG
- SELECT 1 / SWAPDB 0 1
 - Переключить базу; поменять две базы местами
- AUTH пользователь пароль
 - Аутентификация, если настроена

Ключи

- EXISTS ключ / TYPE ключ
 - Есть ли ключ; какого он типа
- DEL ключ / UNLINK ключ
 - Удалить сразу; удалить в фоне — лучше для больших значений
- RENAME старый новый
 - Переименовать
- SCAN 0 MATCH "sess:*" COUNT 100
 - Безопасный обход пространства ключей порциями
- RANDOMKEY / DBSIZE
 - Случайный ключ; сколько ключей в базе

Осторожно: команда KEYS * блокирует сервер на время обхода. На рабочем сервере пользуйтесь SCAN.

Строки и счётчики

- SET ключ значение
 - Записать; EX 60 — со сроком жизни, NX — только если ключа нет
- GET ключ / MGET к1 к2
 - Прочитать одно или несколько значений
- SETNX lock 1
 - Простейшая блокировка: запись только при отсутствии ключа
- INCR / INCRBY / DECR
 - Атомарные счётчики
- APPEND / STRLEN
 - Дописать в конец; длина значения
- GETSET / GETDEL
 - Прочитать и сразу заменить или удалить

Списки

- LPUSH / RPUSH ключ значение
 - Добавить в начало или в конец
- LPOP / RPOP ключ
 - Снять элемент с края
- LRANGE ключ 0 -1
 - Показать весь список
- LLEN / LREM ключ 1 значение
 - Длина; удалить вхождения
- BLPOP ключ 5
 - Ждать элемент до пяти секунд — основа простой очереди задач
- LTRIM ключ 0 999
 - Оставить только первую тысячу — ограничение длины журнала

Множества

- SADD / SREM ключ элемент
 - Добавить, удалить
- SMEMBERS / SCARD
 - Все элементы; их количество
- SISMEMBER ключ элемент
 - Проверка вхождения за постоянное время
- SINTER / SUNION / SDIFF
 - Пересечение, объединение, разность множеств
- SPOP ключ / SRANDMEMBER ключ 3
 - Снять случайный элемент; выбрать несколько без удаления

Упорядоченные множества

- ZADD рейтинг 100 "аня"
 - Элемент с числовым весом
- ZINCRBY рейтинг 5 "аня"
 - Изменить вес
- ZRANGE рейтинг 0 9 WITHSCORES
 - Первая десятка по возрастанию веса
- ZREVRANGE рейтинг 0 9
 - Топ по убыванию — типичная таблица лидеров
- ZRANGEBYSCORE ключ 10 50
 - Выборка по диапазону весов
- ZRANK / ZSCORE / ZCARD
 - Позиция, вес элемента, размер множества

Хеши

HSET user:1 name "Аня" age 30
Поля объекта в одном ключе

HGET user:1 name / **HMGET** user:1 name age
Одно или несколько полей

HGETALL user:1
Все поля и значения

HDEL / **HEXISTS** / **HLEN**
Удалить поле; проверить наличие; число полей

HINCRBY user:1 visits 1
Счётчик внутри хеша

Срок жизни ключей

EXPIRE ключ 300 / **PEXPIRE** ключ 500
Жить 300 секунд / 500 миллисекунд

TTL ключ
Сколько осталось: -1 — без срока, -2 — ключа нет

PERSIST ключ
Снять срок жизни

SET ключ значение EX 60 NX
Запись со сроком и только при отсутствии — так делают блокировки

Сообщения и потоки

SUBSCRIBE канал / **PUBLISH** канал "текст"
Подписка и отправка: сообщения не сохраняются

PSUBSCRIBE "news.*"
Подписка по маске

XADD stream * field value
Поток: записи сохраняются и имеют идентификаторы

XREAD COUNT 10 STREAMS stream 0
Чтение записей потока

XGROUP CREATE / **XREADGROUP** / **XACK**
Группы потребителей с подтверждением обработки

Транзакции и скрипты

MULTI ... **EXEC**
Команды копятя и выполняются одной пачкой

DISCARD
Отменить накопленное

WATCH ключ
Отменить транзакцию, если ключ изменился — оптимистичная блокировка

EVAL "return redis.call('GET', KEYS[1])" 1 ключ
Скрипт на Lua выполняется атомарно

Сохранение и сервер

INFO / **INFO** memory
Сводка о сервере: память, клиенты, статистика

CONFIG GET maxmemory* / **CONFIG** SET maxmemory 2gb
Предел памяти и политика вытеснения

BGSAVE / **LASTSAVE**
Снимок базы в фоне; время последнего успешного

BGREWRITEAOF
Уплотнить журнал команд

MEMORY USAGE ключ
Сколько памяти занимает конкретный ключ

SLOWLOG GET 10
Последние медленные команды

CLIENT LIST / **CLIENT** KILL
Подключения и их разрыв

MONITOR
Поток всех команд — только для отладки, сильно нагружает сервер

Приёмы и оговорки

- Именуйте ключи по схеме `сущность:идентификатор:поле` — так их легко искать и удалять группами.
- Задавайте `maxmemory` и политику вытеснения (`allkeys-lru` для кеша) — иначе сервер упрётся в память.
- Кеш — не хранилище: всё, что нельзя потерять, должно быть и в основной базе.
- Большие значения и длинные списки замедляют операции: разбивайте на части.
- Redis однопоточен по командам: одна тяжёлая команда задерживает всех клиентов.
- Для очередей с гарантиями надёжнее потоки (Streams) с группами, чем списки.